

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий**

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов разворачивания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Общее задание

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)

Задание 2. Написание пользовательских правил фильтрации (L7)

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

Задание 4. Проектирование профилей защиты (Без написания кода)

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
3	DirBuster-1.0-RC1	10003	/api/register	1r/s	3

Ход работы

Задание 1.

1. Выполните отправку вредоносного SQLi-запроса (например, `/?id=1' OR '1'='1`) напрямую к приложению (`http://127.0.0.1:5000`) с помощью `curl` и убедитесь, что приложение его обрабатывает (возвращает код 200 OK).

```
D:\уник\4.1 курс\СМЗК\lab2> curl -i "http://127.0.0.1:5000/?id=1'%20OR%20'1'='1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.10.22
Date: Wed, 07 Oct 2026 08:29:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close
```

2. Повторите тот же SQLi-запрос через защищенный Nginx-периметр (`http://127.0.0.1`).

Убедитесь, что встроенный набор правил OWASP CRS перехватывает атаку, возвращая ошибку 403 Forbidden.

```
D:\уник\4.1 курс\СМЗК\lab2> curl -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 07 Oct 2026 08:38:26 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Задание 2.

1. Выполните отправку легитимного HTTP-запроса, подменив заголовок User-Agent на вредоносный сканер уязвимостей из вашего индивидуального варианта, с помощью `curl -H "User-Agent: ..."`.

```
D:\уник\4.1 курс\СМЗК\lab2> curl -i -H "User-Agent: DirBuster-1.0-RC1" http://127.0.0.1:5000/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.10.22
Date: Wed, 07 Oct 2026 08:44:49 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /
```

2. Напишите пользовательское правило WAF в подготовленном файле nginx/custom.conf с присвоением уникального ID из вашего варианта. Правило должно блокировать запросы, содержащие данный User-Agent.

```
D:\уник\4.1 курс\СМЗК\lab2> echo SecRule REQUEST_HEADERS:User-Agent "@contains DirBuster-1.0-RC1" "id:10003,phase:1,deny,status:403,msg:'Blocked DirBuster Scanner'" >> nginx\custom.conf
```

3. Перезапустите контейнеры (docker-compose restart waf) и убедитесь, что WAF успешно блокирует запрос сканера (код 403).

```
D:\уник\4.1 курс\СМЗК\lab2> curl -i -H "User-Agent: DirBuster-1.0-RC1" http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 07 Oct 2026 08:48:25 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Задание 3.

1. Выполните стресс-тест целевого эндпоинта (согласно варианту) утилитой Apache Bench, отправив 500 запросов в 10 потоков: `ab -n 500 -c 10 http://127.0.0.1<ваш_эндпоинт>`. Зафиксируйте, что сервер успешно обработал все запросы без ошибок (Failed requests: 0).

```
D:\уник\4.1 курс\СМЗК\lab2> docker run --rm --network lab2_default httpd:2.4 ab -n 500 -c 10 http://waf/api/register
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking waf (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests


Server Software:      nginx/1.30.5
Server Hostname:      waf
Server Port:          80

Document Path:        /api/register
Document Length:      43 bytes

Concurrency Level:    10
Time taken for tests:  2.293 seconds
Complete requests:    500
Failed requests:      0
Total transferred:    100000 bytes
HTML transferred:     21500 bytes
Requests per second:  218.09 [#/sec] (mean)
Time per request:     45.852 [ms] (mean)
Time per request:     4.585 [ms] (mean, across all concurrent requests)
Transfer rate:        42.60 [Kbytes/sec] received


Connection Times (ms)
  min   mean[+/-sd] median   max
Connect:    0    0   0.3      0    3
Processing: 10   44   8.0     43   93
Waiting:    9   43   8.0     42   93
Total:      10   45   8.1     43   96


Percentage of the requests served within a certain time (ms)
 50%    43
 66%    47
 75%    48
 80%    50
 90%    54
 95%    59
 98%    64
 99%    71
100%   96 (longest request)
```

2. Сконфигурируйте директивы ограничения интенсивности трафика (`limit_req_zone` и `limit_req`) в файле `nginx/user_nginx.conf`, используя параметры `rate` и `burst` из вашего индивидуального варианта.

```
# RATE LIMITING (Задание 3)
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;

server {
    listen 80;
    server_name localhost;

    location /api/register {
        limit_req zone=mylimit burst=3 nodelay;
        limit_req_status 503;

        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location / {
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

3. Перезапустите контейнер WAF, повторите стресс-тест утилитой `ab` и зафиксируйте блокировку избыточных запросов (сервер должен начать отдавать код 503 Service Unavailable для запросов, превышающих лимит).

```
D:\уник\4.1 курс\СМЗКС\lab2> docker run --rm --network lab2_default httpd:2.4 ab -n 500 -c 10 http://waf/api/register
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking waf (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests

Server Software:      nginx/1.30.5
Server Hostname:      waf
Server Port:          80

Document Path:        /api/register
Document Length:      43 bytes

Concurrency Level:    10
Time taken for tests:  0.563 seconds
Complete requests:    500
Failed requests:       496
    (Connect: 0, Receive: 0, Length: 496, Exceptions: 0)
Non-2xx responses:    496
Total transferred:    183824 bytes
HTML transferred:     97884 bytes
Requests per second:  888.16 [#/sec] (mean)
Time per request:     11.259 [ms] (mean)
Time per request:     1.126 [ms] (mean, across all concurrent requests)
Transfer rate:        318.88 [Kbytes/sec] received

Connection Times (ms)
      min      mean[+/-sd] median   max
Connect:    0       1   0.6      1       4
Processing:  4      10   5.7      7      41
Waiting:    4       9   5.6      6      40
Total:      5      11   5.8      8      44

Percentage of the requests served within a certain time (ms)
 50%    8
 66%   11
 75%   13
 80%   13
 90%   17
 95%   22
 98%   30
 99%   36
100%   44 (longest request)

D:\уник\4.1 курс\СМЗКС\lab2> for /L %i in (1,1,10) do @curl -s -o NUL -w "%{http_code}\n" http://127.0.0.1/api/register
200
200
200
200
503
503
503
503
503
503
```

Запросы сверх `rate=1r/s` и `burst=3` получают 503 Service Unavailable

Задание 4.

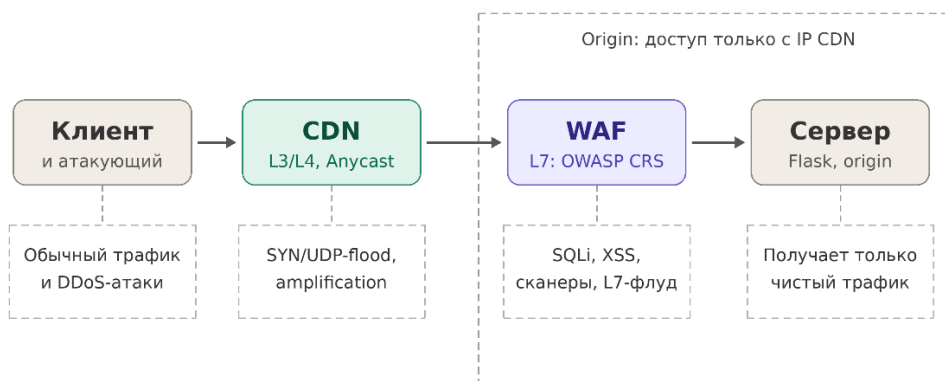
1. Составьте конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (client_body_timeout, client_header_timeout) и обоснуйте свой выбор.

```
client_header_timeout 10s;
client_body_timeout 10s;
keepalive_timeout 15s;
send_timeout 10s;
reset_timedout_connection on;
limit_conn_zone $binary_remote_addr zone=perip:10m;
limit_conn perip 20;
```

Обоснование:

- Slowloris удерживает соединения открытыми, медленно досылая заголовки или тело запроса, и исчерпывает пул worker_connections. Значение по умолчанию (60 с) слишком велико: атакующий успевает занять много соединений.
- client_header_timeout 10s: легитимный клиент передаёт заголовки за доли секунды, и 10 секунд хватает даже при плохой связи. Медленное соединение злоумышленника закрывается, ресурс освобождается.
- client_body_timeout 10s считается между двумя последовательными чтениями тела, а не за всю загрузку, поэтому обычные загрузки не прерываются.
- keepalive_timeout 15s не даёт простаивающим соединениям занимать слоты.
- limit_conn perip 20 ограничивает число одновременных соединений с одного IP-адреса, а reset_timedout_connection on сразу освобождает ресурсы сокета при таймауте.
- Слишком малые значения (1-2 с) привели бы к ложным разрывам у пользователей с медленным каналом, поэтому выбран баланс 10-15 секунд.

2. Спроектируйте архитектурную схему применения CDN (Content Delivery Network) в связке с WAF для защиты от объемных (L3/L4) DDoS-атак.



Принцип эшелонированной защиты:

- Объемные L3/L4-атаки (SYN flood, UDP flood, amplification) гасит CDN: его распределённая инфраструктура и канал значительно больше, чем у сервера организации. Кэш статики дополнительно разгружает сервер.
- WAF отсекает то, что CDN не видит по смыслу: SQL-инъекции, XSS, сканеры уязвимостей, перебор и L7-флуд, а также ограничивает частоту запросов.
- Сервер приложения должен быть скрыт: его IP не публикуется, а фаервол пропускает входящий трафик только с адресов CDN и WAF.
- Чтобы rate limiting на WAF считался по реальным клиентам, а не по IP CDN, настраиваются set_real_ip_from (диапазоны CDN) и real_ip_header X-Forwarded-For.

Вывод: освоила методы развертывания межсетевых экранов прикладного уровня (WAF), написала пользовательские правила фильтрации L7-угроз, а также спроектировала механизмы защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.